

PrivyLayer: Benchmark and Architecture Report

Project: PrivyLayer

Developer: Nithish

Domain: AI Privacy & Data Security

1. Executive Summary

In an era where Large Language Models (LLMs) frequently process raw user inputs containing sensitive data, **PrivyLayer** serves as an essential, lightweight privacy buffer. Built as an open-source Python library, it ensures *zero data exposure* to third-party AI providers. This document outlines the system architecture, core features, and qualitative performance benchmarks of PrivyLayer.

2. Core Workflow & Lifecycle

The lifecycle of a query through PrivyLayer follows a highly optimized, localized loop designed to maintain application speed while ensuring absolute data security:

1. **Input Interception:** The system intercepts raw text containing Personally Identifiable Information (PII) before it leaves the host system.
2. **Detection & Validation:** Identifies entities such as Aadhaar, PAN, phone numbers, and bank details. It incorporates mathematical checksums (like the Verhoeff algorithm) to verify if the inputted identity data is real or fake.
3. **Reversible Masking:** Sensitive data is replaced with structured tokens (e.g., `[[PII_AADHAAR_x1]]`) that retain semantic context for the LLM.
4. **AI Processing:** The external AI model processes the tokenized prompt without ever seeing the raw PII.
5. **Secure Restoration:** PrivyLayer intercepts the AI output locally and seamlessly swaps the token back for the original data before displaying it to the user.

3. Performance & Benchmark Metrics

PrivyLayer is engineered to be a "Zero-Exposure" framework. The following table highlights its performance capabilities and compliance parameters based on its localized design:

Benchmark Metric	System Performance
Latency Overhead	Minimal operational overhead; executes fast, localized pattern matching and checksum validations in real-time.
Network Dependency	None. Localized execution means no data is transmitted to external sanitization servers.
Validation Accuracy	High. Mathematical checksums prevent fake identity numbers from passing through as valid PII.
Data Exposure Risk	0%. Raw sensitive data never leaves the local environment.
Data Loss	0%. Reversible masking perfectly reconstructs original data post-processing.
Compliance Alignment	Structurally aligns with the Digital Personal Data Protection (DPDP) Act via data minimization by default.

4. Speed Test Benchmark

Size	Speed Latency
4 KB	0.8 - 1.2 ms
10 KB	1.7 - 3.2 ms
50 KB	9.10 - 11.12 ms
100 KB	18.2 - 21.6 ms

5. Technical Specifications

- **Language/Ecosystem:** Python (Available via PyPI: `pip install privylayer`)
- **Supported PII Types:** Identity details (Aadhaar, PAN), Contact info (Phones, Emails), Financial data (Bank accounts, IFSC codes).
- **Infrastructure Impact:** Lightweight dependency; operates entirely on the host machine.

6. Conclusion

PrivyLayer stands out as a high-performance, Atmanirbhar (indigenous) privacy shield. By embedding data protection directly into AI workflows, it ensures that developers and organizations can leverage advanced AI tools without compromising citizen data sovereignty or application performance.

7. About Developer

Meet Nithish, the developer behind PrivyLayer - a lightweight, open-source Python privacy library published on PyPI. Passionate about Programming , AI , he engineered the library to dynamically detect and mask sensitive information, even implementing advanced algorithms like the Verhoeff checksum for structural verification.